

STALKER: A Framework to Analyze Fragility of Cryptographic Libraries under Hardware Fault Models

Guorui Xu, Fan Zhang*, Xinjie Zhao, Yuan Chen, Shize Guo, Kui Ren
Zhejiang University, Hangzhou, China

Abstract—For embedded devices, the uncertainty of target physical environments is always a great challenge. With constrained resources and common overloaded uses, they can be more exposed to hardware faults. Other than stability and ordinary security issues, there exist some subtle phenomena that lead to potential cryptanalysis or secret leakage. In this paper, we present STALKER, a framework to analyze the fragility of libraries under hardware fault models. Compared with existing tools, our framework targets faulty execution outputs, and can flexibly work on different libraries, architectures and support different search schemes. We find dozens of security-sensitive bits that may cause critical issues and provide detailed analysis.

Index Terms—Fault injection, Embedded Systems, Cryptographic libraries, Hardware Faults

I. INTRODUCTION

Embedded systems and devices are facing many physical attacks under deployments. Especially, these devices could be usually put into uncertain, even extreme environments, or used overloaded. The above situations all lead to a more dreadful status quo of potential hardware faults on embedded devices. Combined with developing fault attacks that utilize faulty executions, the faults induced by hardware limitations are already great threats to embedded devices. Using fault attacks, retrieving secret keys from interfered executions of cryptographic operations, or bypassing security checks and lift privilege are no new things. For example, by overclocking CPU from system level, the adversary can trigger some hardware faults to break the ARM TrustZone [25]. Invoking normally harmless memory read operations, the adversary can also achieve RowHammer attack and extract keys from faulty ciphertext outputs [29].

Furthermore, some inherent properties of embedded devices, such as being bare-metal and resource-constrained, have determined that only limited security solutions can be applied. It could be a great challenge to defend against potential fault attacks and analyze the weaknesses of target software or firmware. Identifying the vulnerability before the deployments might be the most acceptable way to mitigate such threats. From this direction, many detecting and testing frameworks are proposed. Many applications and operating systems also have runtime self-tests to ensure that the basic functions and core components are not corrupted before actual executions. Static and dynamic analysis tools are utilized to detect guided weaknesses of the software from the source or binary code levels [8], [23]. Fuzzing technology is also a powerful tool to discover potential vulnerabilities related to robustness and security, which can inject inputs and capture abnormal execution results like crashes or memory safety violations.

Though these detection methods already covered most layers that the software involved, they mainly focus on the robustness of programs and conventional security bugs induced by the faults. More domain-specific security properties are generally not involved. Sometimes, the injected fault leads to successful executions, however, can badly damage the security of the target system as well. In these executions, the processes end normally, since there are no critical

violations of memory-safety rules or crash happening, or even the execution can pass some built-in functional tests. Yet some subtle phenomena that the adversary can utilize to perform exploitation. For instance, the execution output could be abnormal and potentially leak information to the adversary.

This kind of fragility of executables under hardware fault models is not investigated thoroughly. Exploring the existence and reasons for such weaknesses is an interesting topic. Moreover, studying the security-sensitive locations in this very field may also help to develop mechanisms that harden the binary executables. These reasons motivate us to develop a unified framework to analyze and check the domain-specific property violations for binary executables under hardware fault models. We choose to consider the cryptographic libraries as our analysis targets because of their essential role played in deployed embedded devices. As for the searching schemes, we focus on the violations of security properties derived from existing cryptanalysis methods.

Our contributions of this paper are summarized as follows:

- 1) We design and implement STALKER, a unified and flexible framework to analyze security-sensitive fragility under hardware fault models targeting binary executables.
- 2) Utilizing searching schemes inspired by formerly proposed cryptanalysis methods, we experiment under the single bit-flip fault model over four widely used cryptographic libraries on x86_64 and ARM64 platforms.
- 3) Using STALKER, we find dozens of security-sensitive bits that cause critical security issues, and provide several case studies that uncover some characteristics of the fragility.

Our source code and related artifacts, including the mutation database of instructions, will be released in the future.

II. BACKGROUND AND RELATED WORK

A. Fault Injection and Fault Attacks

Fault injection is a widely used technology to identify and detect vulnerabilities and exploitation under different fault models. It can be done physically or by simulations [15], [27], either at the software level or hardware level [1], [5]. Extended from fault injection research, studying how the induced faults interfere with the executions of cryptographic algorithms and affect security assurances is a popular and well-studied area, named fault analysis. It can be regarded as a branch of cryptanalysis. The researchers consider the already found fault injection techniques and try to exploit such faults in current deployments of target algorithms to achieve cryptanalysis goals, such as key recovery. For example, differential fault analysis (DFA) assumes the adversary can inject a fault in a specific round of AES computation [28], and then he can recover the master key by analyzing the retrieved faulty/correct ciphertext pairs. In the proposed persistent fault analysis (PFA), the adversary injects a 1-bit fault into the OpenSSL library [29]. The injection changes the value of the non-linear S-box, and then the adversary can recover the master key by utilizing the statistical properties of the output ciphertexts.

*Corresponding author: fanzhang@zju.edu.cn

B. Fault models.

The faults that can be injected into targets are abstracted into different models based on their data representation behaviors, named fault models. For example, bit-flip, stuck-at-zero [26], bit/byte-set, etc. Each fault model can have various ways to trigger. As for the bit-flip model, it can be triggered by software-level actions (i.e., RowHammer attack [19], DVFS fault injections [25]), hardware-level injections (laser fault injections), or even unavoidable environment change (cosmic radiation, etc.). The root cause of a fault occurrence generally lies in the hardware, and many of them cannot be avoided from the software level. Considering its generality, we target the single bit-flip fault model in our experiments, while other fault models can be conveniently implemented in our framework.

C. Fault Vulnerability Detection and Testing

Mutation Testing. A core idea in our paper, *mutants*, is derived from *mutation testing* [13]. Mutant originally means the source code that is modified in small ways, for example, the change of an operator from the expression $a = b + 1$ to $a = b - 1$. To fit our topic targeting binaries, the mutants in this paper are binary instructions other than the source code. Therefore, the mutants are defined as the instructions that are mutated from the original instruction based on the chosen fault model. As for the bit-flip fault model, an example mutant of ARM64 instruction `add w1, w2, w3`, whose binary representation is `0x4100030b`, can be `add w0, w2, w3`, which will be assembled to `0x4000030b`.

Vulnerability detection. Vulnerability detection is another well-studied area. It can be achieved in two directions. From the fault injection part, the goal of the detection is to find the best injection setup and parameters [16], [24]. From the fault target part, the weakest part of the target under specific fault models can be found in well-designed experiments [9], [12]. Many of them use symbolic execution or other dynamic analysis tools to find the security-related vulnerabilities induced by injected faults [8], [23].

Fuzzing. Generally, fuzzing is a bug detection and exploitation technique in software development and testing [17], [18]. It monitors the execution of programs under different inputs and means to find the bugs triggered by unexpected inputs in the normal tests. It has been shown to be very powerful to find bugs like use-after-free, or buffer-overflow [11], [20]. However, the cases for the fault analysis area are something different. The adversaries generally do not care about the failed execution. Instead, many of them care more about the outputs, and try to dig something out of the collected data.

III. METHODOLOGY AND FRAMEWORK DESIGN

In this section, we present our methodology and propose our framework design for STALKER.

A. Threat Model

By investigating several real-world fault injection techniques including RowHammer [19], laser-based fault injection, and common threat model for existing fault analysis, we will assume the following attack model in our framework.

- 1) The adversary can inject different faults based on a fault model into the binary format of the cryptographic library.
- 2) The adversary can hardly control the very precise fault location, but he can target specific memory pages or large segments of where the fault should occur.
- 3) The adversary can observe the outputs and the behaviors of the resultant faulty executions. The executions are originally meant to perform relevant cryptographic operations, such as encryptions, decryptions, and signings.

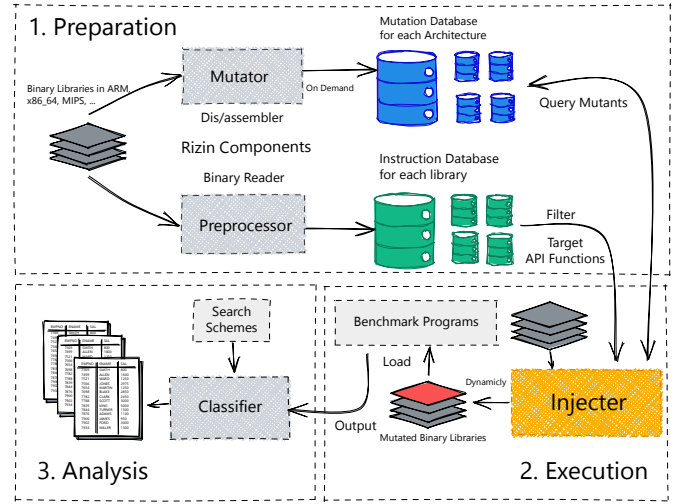


Fig. 1: Framework Overview.

B. Methodology

According to our threat model, the mission of the framework should focus on generating the mutated libraries and analyzing the unexpected outputs of the faulty executions. To achieve this, we follow a workflow that mainly contains three phases, as Fig. 1 shows:

Preparation: In this phase, we collect the required information about the targets, including the instructions and file information of the target libraries. The instructions belonging to target functions are extracted into a database for the generation of their corresponding mutants.

Execution: After the mutation database and instruction database are ready, we can then perform the injections into the target libraries. Since there will be a vast number of mutated libraries, we choose to dynamically generate them and use these libraries when running the benchmark programs. The programs are meant to finish normal tasks. Targeting cryptographic libraries, the payloads are simple programs that perform cryptographic operations. Aggregating the corresponding outputs is also required.

Analysis: To understand if, what and why some executions go wrong after loading the mutated libraries, the outputs will be processed and analyzed in this phase. Targeting different algorithms or properties, different search schemes should be specified and used to identify the possible exploitation and vulnerabilities. Mostly, the search schemes are inspired by the formerly proposed attacks that utilize some unique characteristics of the faulty outputs.

C. Components

We introduce some detailed component designs as follows.

1) Preprocessor

STALKER prepare basic information of the target library, including the architecture, symbol names of the target functions, and instructions and their offsets. We choose the open-sourced Rizin framework [22] to reverse engineer the target binary libraries. Utilizing Rizin framework [22], the analyzed results and information are stored in the database for further processing. Mainly, the database includes the disassembled instructions and their offsets, the recognized symbols and function names, etc. In addition, it can be easily scripted to further extract the required information in the following stages.

2) Mutator

As mentioned in Sec. II, the mutant is a core idea of our design. Since the mutants are at the level of instructions, they can be

generated in advance and stored in a database when reading the instructions from the library. However, generating mutants for *every* possible instruction can be an impossible task due to the vast number of distinct instructions. Therefore, we choose to generate the mutations on demand by specifying target functions and symbols in the binaries. To avoid the flooding of basic runtime errors such as “invalid instruction”, the generation will only preserve the instructions that are meaningful and filter out others. The database will grow more complete as we run more experiments, and each architecture shares the same mutation database.

3) Injector

The component injector is responsible both to inject the mutants into real libraries and to perform the execution of target programs. The number of all possible mutants for a library can also be enormous, therefore the generation of the mutated target library is implemented dynamically. That is to say, when specifying the mutant of a specific instruction of the library, the corresponding binary file will be generated on the fly and then loaded when running the target program. After the execution, the temporary binary file will be cleaned up to preserve disk space.

To accelerate the executions, we implement a parallel executor which can run a batch of experiments and aggregate the outputs at the same time. For Unix-like systems, it will catch stdout and stderr for further detailed diagnosis.

4) Classifier

As shown in Fig. 1, the core component of phase Analysis is Classifier. It accepts the execution outputs and corresponding mutants, then identifies if some outputs are matched by the pre-defined searching schemes. The matches actually indicate the violations to some of our security properties. In the implementation, it contains a parser that extracts data fields from outputs, and a metric calculator that processes the based on different metrics for each result.

Mainly there are four categories for the execution results, including *No Effect*, *Availability*, *Potential Cryptanalysis*, and *Secret Leakage*. They generally describe the severity of the execution results, from lower to higher. *No Effect* category shows that the executions are not affected by such mutants, at least for functional correctness. These results indicate that either the mutated instructions are not accessed in a normal execution, or take no effect on the final results. Mutants classified in this category would be regarded as harmless in a sense. If an execution result is classified into the *Availability* category, it means that the execution is not finished normally, therefore users or monitor systems may be notified. Crashes, buffer-overflows, failing at tests, and other ordinary runtime errors are all classified into the *Availability* category. Since they have been studied thoroughly in other works, we are more interested in the other two categories such that there would be potential information leakage. *Potential Cryptanalysis* category shows that the execution is performed successfully, however some notable phenomena appear that may degrade the security level. *Secret Leakage* category shows that there is indeed some leakage of secrets that occurred in the outputs results of the corresponding executions. To classify the outputs into these two categories, we need to define the searching schemes which be detailed next.

D. Searching Schemes

In our initial implementation, the chosen searching schemes focus on finding some obvious secret leakage and potential cryptanalysis, for example, the following properties:

- 1) check if the outputs have stable probabilities that leak some bytes from plaintexts, round keys, or master keys.
- 2) check if the outputs have an odd byte-level distribution.

- 3) check if the success rate is biased (in some cases, the results show it is functional and correct, while others are not).

These searching schemes are mostly inspired by formerly proposed cryptanalysis and fault attacks that utilize unique characteristics of faulty executions. The full search schemes used in our experiments and their origins are listed in Table I.

TABLE I: Schemes for Experiments.

Category	Scheme	Inspired by
<i>Potential Cryptanalysis</i>	Odd Distribution	Statistical Fault Analysis [7], [29]
	Duplicate Decrypted Duplicate Ciphertexts	Potentially leak key/IV related information since the plaintexts are randomly generated
	Functional Partially Functional Partially Correct	Ineffective fault analysis [2], [4], the faults influence the results with probability.
<i>Secret Leakage</i>	Leak Plaintexts Leak Keys	Trivial
	Nonce Reuse	Nonce Reuse Attack [14]

IV. IMPLEMENTATION AND EXPERIMENTS

A. Experiment Setup

We implement our framework in Rust programming language. Since our target platforms are embedded devices that are dominated by ARM architectures, we run our experiments primarily on an ARM64 Linux platform. To simplify our collecting of the execution outputs, we choose to run the components of the framework on the same machine. In addition, we also provide several cases on x86_64 Linux to show cross-platform common results.

As for the targeted libraries, we choose some widely used cryptographic libraries for embedded systems, including OpenSSL, LibreSSL, mbedTLS, and Libgcrypt. Of course, the experiments can be easily extended to other libraries. The library versions and corresponding commit hashes are listed in Table II.

TABLE II: Experiment Summary on ARM64 Linux Platform.

	OpenSSL	LibreSSL	Libgcrypt	mbedTLS
Versions	v1.1.1q	v3.6.1	v1.9.4	v3.1.0
Commit Hash	29708a	aa1f9b	05422c	d65aeb
Tested Bits	35158	33717	60506	31292
Weak Bits	868	2128	100	905
Critical Bits	107	123	11	77

The target algorithms include the AES software/hardware implementations, block cipher modes operations, and ECDSA. To make the found results more convincing and eliminate false positive cases, the probabilistic characteristics of outputs should also be considered in the classification. The general benchmark program would perform a normal cryptographic operation over pseudo-random data many times (1000 in our experiment setup) to ensure the found results are not coincident. Then STALKER would then process the execution outputs and calculate the metrics to perform classification and produce the report of the found mutants.

B. Experiment Results

1) Overview

The summary results are shown in Table II. Weak bits represent those mutants that are classified into *Potential Cryptanalysis* category, and critical bits are in *Secret Leakage* category. We test more than 160,000 bits in the targeted libraries. In each library, we all find hundreds of weak bits and dozens of critical bits. The detailed counts and percentage of mutants for each library are shown in Table III.

As for the most concerning category, *Secret Leakage*, we summarized the notable cases in Table IV. It includes three classes, which

TABLE III: Summary of Experiment Results.

Algorithm	Lib	# of Inst	Availability		Cryptanalysis		Leakage	
Plain AES-128	OpenSSL	5042	1789	35.48%	786	15.59%	19	0.38%
	LibreSSL	18485	7769	42.03%	1872	10.13%	60	0.32%
	Libgcrypt	46446	40277	86.72%	5	0.01%	0	0.00%
	mbedtls	22860	2678	11.71%	56	0.24%	0	0.00%
CTR Mode	OpenSSL	3668	637	17.37%	39	1.06%	24	0.65%
	LibreSSL	3204	1093	34.11%	252	7.87%	51	1.59%
	Libgcrypt	7009	1032	14.72%	31	0.44%	1	0.01%
	mbedtls	3486	1287	36.92%	626	17.96%	76	2.18%
CBC Mode	OpenSSL	22170	2361	10.65%	40	0.18%	8	0.04%
	LibreSSL	6939	2916	42.02%	4	0.06%	0	0.00%
	Libgcrypt	5825	1244	21.36%	64	1.10%	8	0.14%
	mbedtls	4946	2826	57.14%	223	4.51%	1	0.02%
ECDSA	OpenSSL	4278	2490	58.20%	3	0.07%	56	1.31%
	LibreSSL	5089	1340	26.33%	0	0.00%	12	0.24%
	Libgcrypt	1226	823	67.13%	0	0.00%	2	0.16%
	mbedtls	982	518	52.75%	58	5.91%	0	0.00%

are Leak Keys, Leak Plaintexts, and Nonce Reuse. As their names imply, the resultant consequences may lead to critical secret leakage. In Table IV, the corresponding average ratios that leaked bits take part in the full secret (i.e., the plaintexts or the key) are listed. The “Rate” column represents the nonce reuse rate in a sequence of 1000 times signatures. Many cases (colored with the deepest red) are critical since they have a 100% ratio or rate to leak the secret. In some other cases (colored with lighter red) are not that critical, but can also badly damage the security of the target algorithm.

2) Plain AES-128

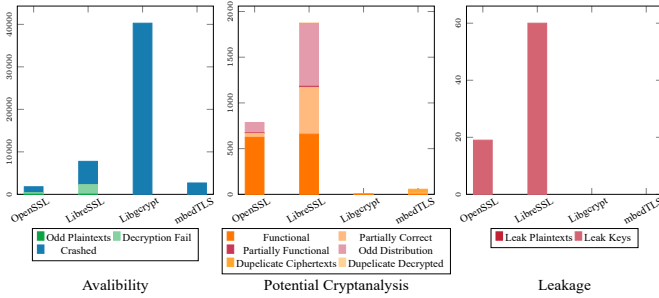


Fig. 2: Results of Plain AES-128 Experiments.

In these experiments, we mainly focus on the software implementations of AES, including the key schedule part and the en/decryption part. Since there might be some platform-specific accelerations, such as Crypto Extensions for ARMv8, we need to disable such abilities for corresponding libraries. The experiment results of different classes for Plain AES-128 experiments are shown in Fig. 2.

As mentioned before, the *Availability* category can be viewed as safe, since they either can be detected with few efforts or crashed and output no useful information. The found mutants in *Secret Leakage* category can leak some bytes of the round keys. In these cases, the leaked key byte length ranges from 2 to 4 bytes. The detailed leaked bytes and the count of corresponding mutants are listed in Table IV. 13 mutants can leak up to 25% (4 bytes) of the round key, which degraded the security level of encryption. Compared with other libraries, Libgcrypt performs better under the single bit-flip fault model by having fewer cases in *Potential Cryptanalysis* and *Secret Leakage* categories.

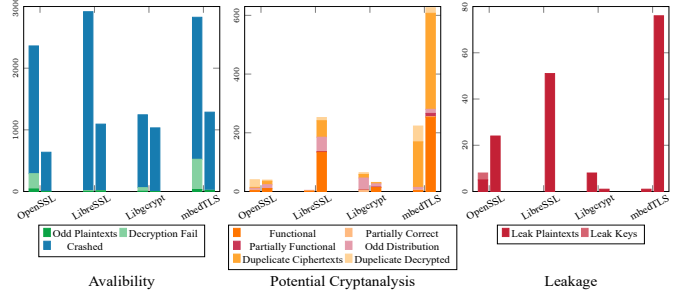
3) Modes Operations

In experiments for Modes Operations, the main consideration is that we can focus on some generally designed functions just like

¹In each stack bar group, the left bar represents CBC experiments, the other one represents CTR experiments.

TABLE IV: Detailed Results for *Secret Leakage* Category.

	Leak Keys		Leak Plaintext		Nonce Reuse	
	Ratio	Count	Ratio	Count	Rate	Count
OpenSSL	10%	9	10%	0	10%	23
	25%	13	25%	0	25%	0
	75%	0	75%	4	75%	5
	100%	0	100%	10	100%	28
LibreSSL	10%	26	10%	2	10%	8
	25%	34	25%	5	25%	1
	75%	0	75%	13	75%	2
	100%	0	100%	31	100%	1
Libgcrypt	10%	0	10%	0	10%	0
	25%	0	25%	1	25%	0
	75%	0	75%	0	75%	0
	100%	0	100%	8	100%	2
mbedtls	10%	0	10%	3	10%	0
	25%	0	25%	11	25%	0
	75%	0	75%	9	75%	0
	100%	0	100%	54	100%	0

Fig. 3: Results of Mode Experiments.¹

work modes. In this way, if an adversary can inject faults into these functions, he can achieve the secret leakage goal regardless of the target cipher choice. We successfully find dozens of cases that leak plaintexts and keys in these experiments. The mutants that leak plaintexts mostly occurred in the CTR experiments. Of all the cases, some of them even leak the *full* 16-byte plaintexts, which means the corresponding executions output the plaintexts as ciphertexts directly. The cases that leak keys have less impact, they also at most leaks 25% of the encryption key.

4) ECDSA

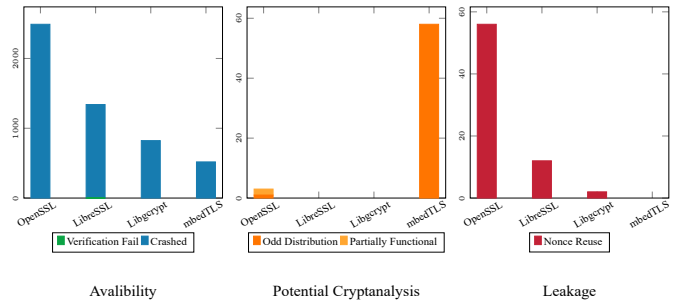


Fig. 4: Results of Plain ECDSA Experiments.

We also experimented over asymmetric algorithms such as ECDSA. To find nonce reuse cases of these faulty executions, we chose to target the nonce generation functions in the libraries. As the result, the bit-flip fault model can indeed trigger some randomness corruption and let the nonce be reused in a sequence of signings. The reuse rates for these mutants range from 0.1% to 100%, as shown in Table IV. While the achieved damage to the cryptosystem is critical enough. If the adversary can collect at least two distinct signatures using the same nonce, he can recover the secret key. Among the

libraries, only mbedTLS has no mutants that lead to nonce reuse.

5) Additional Architectures

To validate our results in platforms other than ARM64 Linux, we experimented OpenSSL and Libgcrypt experiments on x86_64 Linux. The found results are listed in the following table, which shows that the related fragility is common across the architecture.

TABLE V: Experiments on x86_64 Linux platform.

	Leak Ratio	Keys Count	Leak Ratio	Plaintext Count	Nonce Rate	Reuse Count
OpenSSL	12~20%	17	50%	15	75%	5
	25%	2	100%	17	>98.8%	4
Libgcrypt	12.5%	1	75%	0	75~90%	8
	18.6%	1	100%	8	100%	1

C. Case Studies

By analyzing the experiment results, we find several interesting examples that trigger alter conditions monitoring specific secrets or different violations' against pre-defined security properties. In this section, we will present two case studies from previously found mutants and study how the single bit-flip impacts.

1) Round-reduced AES in OpenSSL

In the Functional class, one of the found cases by STALKER mutates the instruction `movz w1 0xa to movz w1 0x2`.

By exploring the assembly codes and source file, we find that this instruction is compiled from the codes that initialize the value of the round number in the key context (`key->rounds`). This very round number will be also used to control the iteration numbers of AES rounds in the encryption. In normal AES-128 encryption, the round number should be `0xa`, which is 10. Changing the round number to `0x2` will lead to a round-reduced computation. Due to some performance optimizations embedded in the AES software implementation, the resultant round number is 4 other than only 2. While a 4-round AES is also proved to be vulnerable [6]. The adversary can easily build distinguishers of ciphertexts or even recover the master key by performing proper cryptanalysis [3], [10].

In the decryption procedure, `AES_set_decrypt_key` directly calls `AES_set_encrypt_key` function, which is exactly the bit-flipped part. In this way, the same initiated `rounds` value will be used, and the decryption would be exactly the inverse of the faulty encryption. Therefore, the faulty ciphertexts can be decrypted to the original plaintexts, and users will not even notice without performing some KATs (Known Answer Tests).

2) Plaintext leakage in CTR Mode

In our experiment results, there are in total of 76 mutants that exist in mbedTLS that can leak plaintexts in CTR work mode. We are very interested in the reasons for these leakages. By analyzing the generated mutants and their original instructions, we classify the mutants by their semantics in Table VI. 42 of 76 found mutants are instructions that mutated from memory load or store instructions. 23 instructions are originally used to branch, whose opcode includes `cmp`, `b.ne`, etc. Other mutants are all mutated from arithmetic instructions.

TABLE VI: Mutants that leak plaintexts in CTR mode of mbedTLS.

Categories	Dominant Args	Count	Examples
Mem.	<code>[sp + 0x68], x0</code> <code>[sp + 0x28], w0</code>	42	<code>ldr x0, [sp, 0x68]</code> <code>str x2, [sp, 0x28]</code>
Branch	<code>x0</code>	23	<code>cmp x0, 0</code> <code>b.ne 0x80</code>
Arith.	<code>w0</code> <code>x0</code>	11	<code>sxtb w0, w0</code> <code>add x0, 0</code>

CTR work mode encrypts nonce, then performs XOR with the plaintexts over the encrypted stream, and finally outputs the results

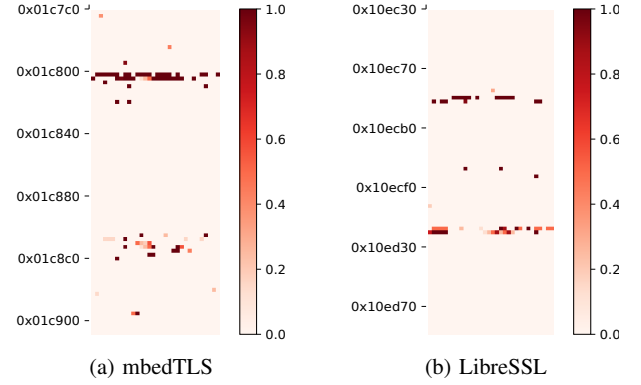


Fig. 5: Heatmap of Leak Plaintexts Class in CTR Mode Experiments.

as ciphertexts. In this way, if the XOR part, or the encrypted stream part, is manipulated, then the produced ciphertexts could partially leak the bytes of plaintexts. Almost all the found mutants impact the encryption process likewise, such as zeroing the encrypted stream part by changing the r/w offset. There is an instruction such that every possible bit-flip leads to plaintext leakage: `ldr x0, [sp, 0x68]` at offset `0x1c804`, which loads data from the stack.

The spatial heatmaps of the Leak Plaintext class for CTR mode in mbedTLS and LibreSSL are shown in Figure 5. Each row contains 32 colored squares, representing a 32-bit instruction in the target region. The results show that the most critical bits are generally clustered as a group, which may be taken advantage by the adversary without precise injection ability.

We note that in these mutants, the arguments of original instructions are dominated by a small set (`x0`, `w0`, etc.). It implies that these registers and memory locations could be very security-sensitive. If their values and manipulations can be guarded or checked before returning or producing any output, these faults could be detected.

D. Findings

General results across libraries. Mostly, the bit-flips affect the availability of the target four libraries. Crashes and functional failures are the most common results in *Availability* category. Mutants that are classified into *Secret Leakage* and *Potential Cryptanalysis* categories are also found in every target library under the single bit-flip fault model. Some mutants would lead to critical security issues, resultant consequences include full key leakage and full plaintext leakage. This fragility is kind of unavoidable by normal software hardening techniques. For example, OpenSSL uses a carefully designed DSA nonce generation function `BN_generate_dsa_nonce`, which utilizes a hash function to improve the randomness. However, it can still be influenced by the single bit-flip fault model.

Error handling and Know Answer Tests (KATs). We find it interesting that Libgcrypt has much more crashed results compared with other libraries. By analyzing the stderr files generated from the executions, we find that it is because KATs are enabled for Libgcrypt by default. Before the real execution, Libgcrypt will invoke a self-test to ensure that the loaded binary can perform some operations normally. Moreover, the internal exception handlers are also very sound. For example, it can detect unintended data mutation of an immutable big number. The results indicate that the code quality of Libgcrypt is better from a point of view, and the security considerations are more complete than other libraries. In this way, Libgcrypt would be more confident to detect and escape from the injected faults.

Hardware Accelerations. We also target other hardware implementations for plain AES, such as `aesni`, SSE3, etc. on x86_64 Linux, and `armv8_ce` on ARMv8 Linux. In general, the results show that these accelerated implementations are much less vulnerable under the single bit-flip fault model. Though some mutants could lead to potential cryptanalysis, there exists no found mutants are classified into the Leakage category under our searching schemes.

V. DISCUSSION

A. Possible Mitigations

We have discussed in Sec. IV-D that the runtime tests, especially KATs, can effectively detect unexpected behaviors and warn the users. For example, the current kernel crypto API will run the self-tests to ensure the cryptographic modules are not corrupted when powering up [21]. At the software level, other ways include regularly checking the signed digest of binary libraries. Hinted by the result analysis from Sec. IV-C2, the sensitive registers and memory locations seem limited. Protecting these registers and guard at the found critical paths to harden the system against the bit-flip faults can be a possible mitigation. At the hardware level, mechanisms like dual-module redundancies and specially designed hardware units can also make a difference. Of course, to fully defend against such aggressive attacks is not realistic and not possible, we can only balance the cost and the benefit.

B. Limitations and Future Work

The first limitation of our work is that it currently only supports single bit/byte fault models. To support the multi-bit fault models is convenient, extending the *Mutator* part in our framework is sufficient, while it will introduce a much larger mutation database. Supporting the dynamic and transient fault model requires more effort since the current workflow modifies the library, which will be copied into memory during the execution.

Another limitation is that we only focus on the cryptographic libraries, and the searching schemes are not complete enough. However, the workflow and the proposed framework still make a difference in the detection of fault-related vulnerabilities. By extending the models, we can also target other suitable libraries, such as the libraries that are used in critical environments or systems with strong fault resistance requirements.

VI. CONCLUSION

To identify domain-specific security issues caused by faulty executions, we propose a new framework STALKER to analyze the fragility of binary libraries under hardware fault models. By defining the searching schemes inspired by historical cryptanalysis methods, our framework successfully finds dozens of mutants that can leak secrets under the single bit-flip fault model regardless of the architecture. The experiment results of different algorithms and libraries show that the fragility under hardware faults widely exists in the current code base. We also confirm the weaknesses indeed damage the security by further analysis and provide several case studies that uncover the spatial and semantic characteristics of the found mutants.

ACKNOWLEDGMENT

This work was supported in part by National Key R&D Program of China (2020AAA0107700), by National Natural Science Foundation of China (62227805, 62072398), by SUTD-ZJU IDEA Grant for visiting professors (SUTD-ZJUVP201901), by Alibaba-Zhejiang University Joint Institute of Frontier Technologies, by National Key

Laboratory of Science and Technology on Information System Security (6142111210301), by State Key Laboratory of Mathematical Engineering and Advanced Computing, and by Key Laboratory of Cyberspace Situation Awareness of Henan Province (HNTS2022001).

REFERENCES

- [1] A. Benso *et al.*, *Fault injection techniques and tools for embedded systems reliability evaluation*. Springer Science & Business Media, 2003, vol. 23.
- [2] C. Clavier, "Secret external encodings do not prevent transient fault analysis," in *Int. Workshop on CHES*. Springer, 2007, pp. 181–194.
- [3] H. Demirci *et al.*, "A meet-in-the-middle attack on 8-round AES," in *International Workshop on FSE*. Springer, 2008, pp. 116–126.
- [4] C. Dobraunig *et al.*, "Statistical ineffective fault attacks on masked aes with fault countermeasures," in *ASIACRYPT*. Springer, 2018, pp. 315–342.
- [5] L. Feinbube *et al.*, "Software fault injection: A practical perspective," in *Dependability Engineering*. IntechOpen, 2017.
- [6] N. Ferguson *et al.*, "Improved cryptanalysis of Rijndael," in *International Workshop on FSE*. Springer, 2000, pp. 213–230.
- [7] T. Fuhr *et al.*, "Fault attacks on aes with faulty ciphertexts only," in *2013 FTDC*. IEEE, 2013, pp. 108–118.
- [8] T. Given-Wilson *et al.*, "An automated and scalable formal process for detecting fault injection vulnerabilities in binaries," *Concurrency and Computation: Practice and Experience*, vol. 31, no. 23, p. e4794, 2019.
- [9] —, "Combined software and hardware fault injection vulnerability detection," *Innov. Syst. Softw. Eng.*, vol. 16, no. 2, pp. 101–120, 2020.
- [10] D. Gstir, *Analysis of recent attacks on AES*. na, 2012.
- [11] I. Haller *et al.*, "Dowser: a guided fuzzer to find buffer overflow vulnerabilities," in *USENIX 13*, 2013, pp. 49–64.
- [12] N. Jafri, "Formal fault injection vulnerability detection in binaries: a software process and hardware validation," Ph.D. dissertation, Université Rennes 1, 2019.
- [13] Y. Jia *et al.*, "An analysis and survey of the development of mutation testing," *IEEE Trans. Softw. Eng.*, vol. 37, no. 5, pp. 649–678, 2010.
- [14] D. Johnson *et al.*, "The elliptic curve digital signature algorithm (ecdsa)," *Int. J. Inf. Secur.*, vol. 1, no. 1, pp. 36–63, 2001.
- [15] M. Kooli *et al.*, "A survey on simulation-based fault injection tools for complex systems," in *2014 9th IEEE DTIS*. IEEE, 2014, pp. 1–6.
- [16] A. Maldini *et al.*, "Genetic algorithm-based electromagnetic fault injection," in *FDTC*. IEEE, 2018, pp. 35–42.
- [17] B. Miller *et al.*, "The relevance of classic fuzz testing: Have we solved this one?" *IEEE Trans. Softw. Eng.*, 2020.
- [18] B. P. Miller *et al.*, "An empirical study of the reliability of UNIX utilities," *Communications of the ACM*, vol. 33, no. 12, pp. 32–44, 1990.
- [19] O. Mutlu, "The RowHammer problem and other issues we may face as memory becomes denser," in *DATE*. IEEE, 2017, pp. 1116–1121.
- [20] M.-D. Nguyen *et al.*, "Binary-level directed fuzzing for Use-After-Free vulnerabilities," in *RAID*, 2020, pp. 47–62.
- [21] NIST, "Red hat enterprise linux kernel crypto api cryptographic module v4.0," <https://csrc.nist.gov/CSRC/media/projects/cryptographic-module-validation-program/documents/security-policies/140sp2742.pdf>, 2016.
- [22] R. Organization, "Rizin," <https://github.com/rizinorg/rizin>, 2022.
- [23] K. Pattabiraman *et al.*, "SymPLIFIED: Symbolic program-level fault injection and error detection framework," in *2008 IEEE International Conference on DSN*. IEEE, 2008, pp. 472–481.
- [24] S. Picek *et al.*, "Fault injection with a new flavor: Memetic algorithms make a difference," in *COSADE*. Springer, 2015, pp. 159–173.
- [25] P. Qiu *et al.*, "Voltjockey: Breaching trustzone by software-controlled voltage manipulation over multi-core frequencies," in *Proceedings of the 2019 ACM SIGSAC CCS*, 2019, pp. 195–209.
- [26] N. H. Seong *et al.*, "SAFER: Stuck-at-fault error recovery for memories," in *MICRO-43*. IEEE, 2010, pp. 115–124.
- [27] C. Shepherd *et al.*, "Physical fault injection and side-channel attacks on mobile devices: A comprehensive analysis," *Computers & Security*, vol. 111, p. 102471, 2021.
- [28] M. Tunstall *et al.*, "Differential fault analysis of the advanced encryption standard using a single fault," in *IFIP international workshop on information security theory and practices*. Springer, 2011, pp. 224–233.
- [29] F. Zhang *et al.*, "Persistent fault analysis on block ciphers," *IACR TCHES*, pp. 150–172, 2018.